

DEPARTMENT OF INFORMATION TECHNOLOGY — ADVANCED PYTHON PROGRAMMING LAB

LABORATORY EXAMINATION: STUDENT GRADE & ASSESSMENT MODULE

CONTINUOUS EVALUATION & TRANSCRIPT PERSISTENCE SYSTEM

Duration: 2 Hours | Maximum Marks: 40 | Target Topic: JSON File Persistence & Modular Analytics

1. Context & Problem Scenario

The Academic Council requires a resilient command-line utility to record, evaluate, and track student grades across technical training modules. Your assignment is to engineer a modular Python application that manages student performance records in memory and reliably synchronizes them with an external **JSON storage file** (students.json). The program must offer automated grade evaluation, comprehensive search filters, formatted cohort summaries, and defensible error handling.

2. System Architecture & Data Model

The application maintains active records inside a Python **list of dictionaries**. Each record adheres strictly to the schema shown below:

```
students = [
    {"id": 1, "name": "Aarav Sharma", "course": "Python Core", "marks": 88.5, "grade": "A"},
    {"id": 2, "name": "Diya Patel", "course": "Data Science", "marks": 74.0, "grade": "B"}
]
```

Automated Grade Evaluation Logic:

- Marks $\geq 85.0 \rightarrow$ Grade **A** | • Marks ≥ 70.0 and $< 85.0 \rightarrow$ Grade **B**
- Marks ≥ 50.0 and $< 70.0 \rightarrow$ Grade **C** | • Marks $< 50.0 \rightarrow$ Grade **F** (Fail)

Sample Cohort Dataset (For Testing & Initial Seeding):

ID	Candidate Name	Course / Module	Marks (100)	Awarded Grade
1	Aarav Sharma	Python Core	88.50	A
2	Diya Patel	Data Science	74.00	B
3	Rohan Nair	Web Architecture	45.00	F
4	Sneha Kulkarni	Python Core	92.00	A
5	Amit Verma	Data Science	63.50	C

3. Operational Modules & Functional Requirements

Implement an interactive, menu-driven CLI loop displaying the following choices:

[1] Enroll Student | [2] Cohort Directory | [3] Query Records | [4] Revise Evaluation | [5] Purge Record | [6] Save to JSON | [7] Load from JSON | [8] Terminate

Module A: Student Enrollment & Evaluation Engine

- **Candidate Ingestion:** Prompt user for student name, enrolled course/module, and numerical marks obtained (0.0 to 100.0).
- **Identifier & Grade Generation:** Automatically calculate unique incremental id and assign letter grade (A/B/C/F) using the evaluation logic.
- **Registration:** Construct the record dictionary and append it into the master cohort list.

Module B: Cohort Reporting & Search System

- **Formatted Cohort View:** Print all student records in a structured tabular grid with fixed-width column alignments (refer to formatting conventions in main.py). Handle empty cohort state gracefully.
- **Multi-Criteria Lookup:** Allow searching either by unique **Student ID** (exact match) OR by **Course Name / Student Name** (case-insensitive substring match). Display matching candidate cards or print a clear "No records found" alert.

Module C: Record Mutation & JSON Persistence

- **Marks & Course Revision:** Locate student by ID; allow updating candidate name, course, or examination marks. If marks are altered, automatically recompute and update the letter grade.
- **Record Deletion:** Search for student ID, display details, and prompt for confirmation (y/n) before purging from memory.
- **JSON Serialization (Save):** Serialize the cohort list to students.json using Python's json.dump() with indent=4 formatting.
- **JSON Deserialization (Load):** Read students.json using json.load() to restore in-memory records. Handle missing file situations without crashing.

4. Data Integrity & Exception Handling Directives

- **Field Validations:** Name and Course strings must not be blank after trimming whitespace. Examination marks must satisfy $0.0 \leq \text{marks} \leq 100.0$.
- **Automated Bounds:** The letter grade must be determined programmatically by code; manual overrides by user input are prohibited.
- **Robust Exception Trapping:** Wrap numerical user inputs (choice, ID, marks) inside try-except ValueError blocks. Wrap file I/O operations inside try-except (FileNotFoundError, json.JSONDecodeError) handlers. The CLI session must never terminate abnormally due to bad user input.

5. Milestone-Based Evaluation Scheme (40 Marks)

Milestone 1	Interactive CLI Controller, State Management & Ingestion Module	8
Milestone 2	Cohort Tabular Reporting & Multi-Criteria Query Functions	10
Milestone 3	Record Mutation, Deletion Routine & JSON Persistence Sync	12
Milestone 4	Defensive Programming, Schema Constraints & Exception Resilience	10
Total	Comprehensive Laboratory Assessment	40